Portfolio Ranking and Conviction Score Assignment

Background

In systematic investment workflows we often start with a universe of **candidate portfolios**. The goal is to determine which portfolios are more attractive relative to others.

Instead of predicting values, it is often sufficient to **rank portfolios correctly** according to their future performance.

In addition to ranking, portfolio construction often requires **conviction scores** — numerical measures of how strongly we believe in each portfolio relative to the others.

In this task you will build a small pipeline to produce conviction scores for a set of portfolios.

The main goals of this task are to assess whether you can:

- work with Object-Oriented code,
- build a small code pipeline,
- perform simple research on designing conviction scores.

Task

You will be given a small set of Python modules that define the following objects:

- Instrument
- PortfolioPayload
- Portfolio
- PortfolioUniverse

These objects define how candidate portfolios are represented and constructed.

You should **use these objects directly** rather than creating your own representation of portfolios.

Some helper methods are also provided, such as:

- make()
- make_many()
- resolve_portfolios_inplace()

Features

Use yfinance to download daily **OHLCV** data for all stocks appearing in the portfolios.

Construct the following stock-level features:

- 20-day momentum
- 60-day momentum
- 20-day volatility
- 60-day volatility
- 20-day average volume

Momentum can be calculated using simple returns based on adjusted close prices.

Example:

momentum_20 = price_t / price_(t-20) – 1

Volatility can be calculated as the rolling standard deviation of daily returns.

Model Training

Train (at least) one ranking-based models, e.g.,

- **XGBoost using a ranking objective (e.g., pairwise ranking loss)**

The models should output a score used to **rank the portfolios**.

The evaluation target is:

future 20-day portfolio Sharpe ratio

The optimization target should not be to predict the exact Sharpe ratio, but to **produce rankings that align well with future Sharpe**.

Conviction Scores

Ranking models typically produce **scores** that order portfolios by expected performance.

However, portfolio construction often requires **conviction scores** that determine how strongly each portfolio should influence the final allocation.

What is a Conviction Score?

A conviction score represents **how strongly we believe in a portfolio relative to the others**.

For example:

- a portfolio with very strong predicted performance may receive a high conviction score,
- portfolios with weaker signals may receive lower conviction scores.

Conviction scores are often used as inputs to portfolio construction (e.g., Black-Litterman).

Therefore, the mapping from model outputs to conviction scores is an important design decision.

Using the model outputs, implement at least 4-5 methods to compute conviction scores.

Some examples are:

Example 1 — Signal-to-Noise Ratio

Construct a conviction score based on the ratio between:

- predicted ranking scores
- estimated portfolio volatility

This approach attempts to measure the strength of the signal relative to risk.

Example 2 — Percentile Mapping

Use the percentile of predicted ranking score as the conviction score.

Conviction scores should ideally be **comparable across time and across portfolios**.

Analysis

Compare the conviction score methods.

Discuss briefly:

- how the distributions of scores differ,
- whether some methods produce highly concentrated conviction signals,
- how stable the scores appear over time,
- whether higher conviction scores correspond to stronger realized performance.

Code Structure

Your implementation should follow **object-oriented programming principles**.

You must use the provided portfolio objects.

You may add additional classes if useful (for example for dataset, ranking, or conviction score calculations).

Deliverables

Submit:

1. A **Jupyter notebook** describing your workflow and results
2. Supporting **Python modules** for reusable code
3. A short discussion of the conviction score methods